

SLOT6:

Rest_Restfull API, URL Endpoint

Khác biệt giữa REST API và RESTful API

Các nguyên tắc của REST API được đặt ra bởi Roy Fielding trong luận án tiến sĩ của ông vào năm 2000. REST (REpresentational State Transfer) là một **kiến trúc phần mềm** cho việc xây dựng các dịch vụ web. Một RESTful API là API tuân thủ **6 nguyên tắc cốt lõi** sau:

✓ 6 nguyên tắc của REST API

1. Client-Server (Máy khách - Máy chủ tách biệt)

- Máy khách (client) và máy chủ (server) phải được **tách biệt rõ ràng**.
- Client chỉ quan tâm đến giao diện và cách hiển thị dữ liệu.
- Server chịu trách nhiệm **xử lý logic nghiệp vụ và lưu trữ dữ liệu**.

✦ *Lợi ích:* Dễ dàng phát triển song song client và server; tái sử dụng server cho nhiều nền tảng khác nhau.

2. Stateless (Không lưu trạng thái)

- Mỗi yêu cầu từ client đến server phải **chứa đủ thông tin cần thiết để xử lý**.
- Server **không lưu trạng thái phiên làm việc của client**.

✦ *Ví dụ:* Nếu người dùng đăng nhập, token xác thực (JWT, sessionID...) phải gửi trong mỗi request – server không nhớ client nào đã đăng nhập trước đó.

✦ *Lợi ích:* Hệ thống dễ mở rộng (scalable) và đơn giản hơn.

3. Cacheable (Có thể lưu vào bộ nhớ đệm)

- Server nên cho phép client cache dữ liệu để **giảm tải server và tăng hiệu năng**.
- Server phải gửi thông tin về khả năng cache qua các **HTTP headers** (như Cache-Control).

✦ *Ví dụ:* Khi lấy danh sách sản phẩm, response có thể kèm header `Cache-Control: max-age=3600`.

4. Uniform Interface (Giao diện thống nhất)

Là nguyên tắc **cốt lõi** của REST, đảm bảo rằng API có cấu trúc rõ ràng và nhất quán.

Các yếu tố chính gồm:

- **Resource URI:** Tài nguyên nên được biểu diễn bằng **danh từ** (ví dụ `/users/1`).
- **HTTP Methods:**
 - GET → Lấy dữ liệu

- POST → Tạo mới
- PUT/PATCH → Cập nhật
- DELETE → Xóa
- **Định dạng dữ liệu:** JSON hoặc XML (thường là JSON).
- **Dùng mã HTTP chuẩn:**
 - 200 OK
 - 201 Created
 - 400 Bad Request
 - 401 Unauthorized
 - 404 Not Found
 - 500 Internal Server Error

5. Layered System (Hệ thống phân tầng)

- REST cho phép hệ thống được **thiết kế theo nhiều tầng** (layer) như: load balancer, cache, bảo mật, server chính...
- Client không cần biết mình đang giao tiếp với tầng nào.

✦ *Ví dụ:* Client gửi request đến load balancer, nó sẽ chọn một server thích hợp xử lý request.

6. Code on Demand (Không bắt buộc – Mã thực thi theo yêu cầu)

- Server có thể gửi mã (JavaScript, applet...) để client thực thi.
- **Nguyên tắc duy nhất tùy chọn** trong REST.

✦ *Ví dụ:* Server trả về một đoạn JS để client xử lý dữ liệu động – nhưng rất hiếm dùng trong API hiện đại.

Tóm tắt 6 nguyên tắc REST

Nguyên tắc	Ý nghĩa chính
Client-Server	Phân tách giao diện và xử lý dữ liệu
Stateless	Mỗi request độc lập, không lưu trạng thái
Cacheable	Hỗ trợ lưu cache để tăng hiệu năng
Uniform Interface	Giao diện thống nhất, dùng HTTP chuẩn
Layered System	Cho phép thiết kế phân tầng
Code on Demand	Gửi mã thực thi (tùy chọn)

SỰ KHÁC BIỆT GIỮA REST API VÀ RESTFUL API

Sự khác biệt giữa **REST API** và **RESTful API** trong Node.js (và trong phát triển web nói chung) là một trong những điều khiến nhiều người học dễ nhầm. Tuy nhiên, sự khác biệt giữa hai thuật ngữ này **không quá lớn**, và đôi khi chúng được dùng **gần như thay thế nhau** trong thực tế. Dưới đây là cách phân biệt rõ ràng:

1. REST API là gì?

REST API là viết tắt của:

- **REpresentational State Transfer – Application Programming Interface**
- Đây là **một kiểu kiến trúc API** được thiết kế dựa trên nguyên tắc REST.

 Một API được gọi là REST nếu nó:

- Sử dụng **HTTP methods** (GET, POST, PUT, DELETE) đúng cách.
- Có **resource URL rõ ràng** (danh từ, không phải động từ).
- Không lưu trạng thái (stateless).
- Trả dữ liệu qua **JSON hoặc XML**.
- Hỗ trợ **HTTP status codes** đúng chuẩn (200, 404, 201, 400, 500,...).

 Ví dụ REST API:

bash

GET /products	→ lấy danh sách sản phẩm
GET /products/1	→ lấy thông tin sản phẩm ID = 1
POST /products	→ tạo sản phẩm mới
PUT /products/1	→ cập nhật sản phẩm ID = 1
DELETE /products/1	→ xóa sản phẩm ID = 1

2. RESTful API là gì?

RESTful API là:

- **Một API tuân thủ chặt chẽ các nguyên tắc của REST.**
- Thuật ngữ “RESTful” nhấn mạnh rằng API **được thiết kế đúng với tư duy REST**.

Nói cách khác:

-  Mọi **RESTful API** là **REST API**,
-  Nhưng không phải mọi **REST API** đều là **RESTful**.

-  Một API chỉ gọi là **REST API** nếu nó “gần đúng” với REST,
-  Một API gọi là **RESTful** khi nó **tuân thủ nghiêm ngặt** các nguyên tắc REST.

So sánh REST API vs RESTful API

Tiêu chí	REST API	RESTful API
Kiến trúc	Dựa trên nguyên tắc REST	Tuân thủ đầy đủ nguyên tắc REST
Tính đúng chuẩn	Có thể không hoàn toàn đúng REST	Đúng chuẩn REST
Tên gọi tài nguyên	Có thể dùng động từ hoặc danh từ	Chỉ dùng danh từ đại diện cho resource
Sử dụng HTTP method	Có thể không đúng	Sử dụng đúng (GET, POST, PUT, DELETE)
Phản hồi & mã trạng thái	Có thể thiếu logic	Trả về mã trạng thái HTTP chính xác

Trong Node.js

Giả sử bạn dùng Express:

```
js

const express = require('express');
const app = express();

// RESTful: lấy danh sách users
app.get('/users', (req, res) => {
  res.json([ { id: 1, name: 'John' } ]);
});

// RESTful: lấy user theo ID
app.get('/users/:id', (req, res) => {
  const userId = req.params.id;
  res.json({ id: userId, name: 'Jane' });
});
```

Không RESTful:

```
js

app.get('/getAllUsers', ...); // dùng động từ trong route → sai
app.post('/createUser', ...); // không nên dùng động từ trong URL
```

Tổng kết

REST API	RESTful API
Tổng quan kiến trúc API theo REST	API được thiết kế tuân thủ đầy đủ REST
Thường dùng rộng rãi	Chính xác và đúng chuẩn hơn
Có thể lỏng lẻo	Ràng buộc nghiêm ngặt

 Nếu bạn thiết kế API trong Node.js, hãy hướng tới RESTful API để có tính nhất quán, dễ mở rộng, dễ bảo trì.

URL ENDPOINT API CHUẨN (TRONG RESTFUL API)

✓ Cấu trúc chuẩn của một endpoint API

bash

```
[HTTP METHOD] /api/{resource}/{resourceId?}
```

Thành phần	Ý nghĩa
HTTP METHOD	GET, POST, PUT, PATCH, DELETE
/api/	Prefix để định nghĩa API rõ ràng
{resource}	Danh từ số nhiều đại diện cho tài nguyên
{resourceId?}	ID duy nhất để xác định tài nguyên cụ thể

✿ Ví dụ Endpoint API chuẩn với resource users

Tác vụ	HTTP Method	Endpoint	Mô tả
Lấy danh sách người dùng	GET	/api/users	Trả về tất cả người dùng
Lấy người dùng theo ID	GET	/api/users/:id	Trả về 1 người dùng theo ID
Tạo mới người dùng	POST	/api/users	Tạo người dùng mới
Cập nhật toàn bộ người dùng	PUT	/api/users/:id	Cập nhật toàn bộ thông tin
Cập nhật 1 phần thông tin	PATCH	/api/users/:id	Cập nhật một phần (ví dụ chỉ tên)
Xóa người dùng	DELETE	/api/users/:id	Xóa người dùng theo ID

🔗 Quy tắc thiết kế Endpoint API chuẩn

1. Dùng danh từ số nhiều cho resource

- ✓ /api/users (đúng)
- ✗ /api/user (không nên – không rõ là 1 hay nhiều)

2. Không dùng động từ trong URI

- ✓ POST /api/users → Tạo mới
- ✗ /api/createUser → Sai nguyên tắc REST

3. Dùng HTTP method để xác định hành động

Hành động	HTTP method
Đọc	GET
Tạo mới	POST
Cập nhật	PUT/PATCH
Xóa	DELETE

4. Trả về status code chuẩn

- 200 OK – Thành công
- 201 Created – Tạo thành công
- 204 No Content – Xóa thành công
- 400 Bad Request – Dữ liệu sai
- 401 Unauthorized – Chưa đăng nhập
- 404 Not Found – Không tìm thấy tài nguyên
- 500 Internal Server Error – Lỗi server

5. Hỗ trợ phân trang, lọc, tìm kiếm (optional)

- /api/users?page=2&limit=10
- /api/products?category=shoes&sort=price

 Ví dụ thực tế với Node.js + Express
js

```
const express = require('express');
const router = express.Router();

router.get('/users', userController.getAllUsers);           // Lấy danh sách
router.get('/users/:id', userController.getUserById);      // Lấy 1 người
router.post('/users', userController.createUser);          // Tạo mới
router.put('/users/:id', userController.updateUser);       // Cập nhật toàn bộ
router.delete('/users/:id', userController.deleteUser);    // Xóa

module.exports = router;
```

 Tổng kết

Một endpoint API chuẩn cần:

- Dùng HTTP method đúng.
- Đường dẫn là danh từ số nhiều.
- Không chứa động từ trong URI.
- Trả về status code rõ ràng.
- Hỗ trợ phân trang, lọc nếu cần.